

บทที่ 4: การจัดการเหตุการณ์และการรับข้อมูลจากผู้ใช้

1. อธิบายความสำคัญของการจัดการเหตุการณ์จากผู้ใช้ต่อประสิทธิภาพของแอปพลิเคชัน หากไม่มีการจัดการที่ดีจะเกิดผลกระทบอะไรบ้าง พร้อมยกตัวอย่างโค้ดหรือสถานการณ์จริง
2. วิเคราะห์ข้อจำกัดของการรับข้อมูลผ่านแป้นพิมพ์ และเสนอแนวทางแก้ไข
3. อธิบายข้อดีและข้อจำกัดของการรับข้อมูลผ่านเมาส์ พร้อมยกตัวอย่างการใช้งานจริง
4. อธิบายข้อดีและข้อจำกัดของการรับข้อมูลผ่านหน้าจอสัมผัส พร้อมยกตัวอย่างการใช้งานจริง
5. อธิบายข้อดีและข้อจำกัดของการรับข้อมูลผ่านเสียง พร้อมยกตัวอย่างการใช้งานจริง
6. อธิบายความแตกต่างระหว่าง Notification และ Dialog ในการสื่อสารกับผู้ใช้ พร้อมยกตัวอย่างการใช้งานจริง
7. อธิบายการใช้ Notification แบบชั่วคราว (Snackbar/Toast) และผลกระทบต่อประสบการณ์ผู้ใช้
8. อธิบายข้อดีของการตรวจสอบข้อมูลฝั่งผู้ใช้ (Client-side Validation) และผลกระทบต่อประสิทธิภาพระบบ

บทที่ 5: การออกแบบฟอร์มและการจัดเก็บข้อมูล

1. อธิบายหลักการออกแบบฟอร์มที่ดี และผลกระทบต่อประสบการณ์ผู้ใช้
2. ยกตัวอย่างการใช้งานฟอร์มเข้าสู่ระบบ (Login Form) และอธิบายความสำคัญในการตรวจสอบตัวตน
3. วิเคราะห์ข้อจำกัดของการจัดเก็บข้อมูลแบบชั่วคราว และเสนอแนวทางแก้ไข
4. อธิบายการใช้หน่วยความจำชั่วคราว และผลกระทบเมื่อระบบหยุดทำงานหรือปิดเครื่อง
5. อธิบายการใช้ตัวแปรแบบโกลบอล และผลกระทบต่อการทำงานของโปรแกรม
6. อธิบายโครงสร้างข้อมูลที่ใช้เก็บหลายค่าในตัวแปรเดียว พร้อมยกตัวอย่างการใช้งานจริง
7. อธิบายการจัดเก็บข้อมูลภายในอุปกรณ์ และผลต่อการทำงานของแอปพลิเคชันเมื่อไม่มีอินเทอร์เน็ต
8. อธิบายการใช้ Local Storage ในแอปพลิเคชัน พร้อมยกตัวอย่างการใช้งานจริง
9. อธิบายการใช้ TinyDB ใน MIT App Inventor สำหรับการจัดเก็บข้อมูลภายใน

บทที่ 6: การจัดวางองค์ประกอบ UI และ Layout

1. อธิบายหลักการออกแบบ UI โดยใช้ White Space และผลกระทบต่อประสบการณ์ผู้ใช้
2. อธิบายการใช้ Column Widget ในการจัดเรียงวิดเจ็ต พร้อมยกตัวอย่างการใช้งานจริง
3. อธิบายความหมายของ MainAxisAlignment.spaceEvenly และ CrossAxisAlignment.stretch ใน Column Widget พร้อมยกตัวอย่างการใช้งานจริง
4. อธิบายความหมายของ CrossAxisAlignment.end และผลต่อการจัดวางวิดเจ็ต
5. อธิบายการใช้ Feedback Components เช่น Progress Bar ในการแสดงความคืบหน้า
6. อธิบายการใช้ BoxShape.circle ใน Container และผลลัพธ์ที่ได้
7. อธิบายการเปลี่ยน scrollDirection เป็น Axis.horizontal และผลต่อการเลื่อนหน้าจอ
8. อธิบายการใช้ Row ซ้อนใน Column และประโยชน์ในการจัดวางวิดเจ็ตหลายแนวพร้อมกัน

บทที่ 7: (Column, Row, Stack Widget)

1. Column Widget ใช้สำหรับอะไร
2. วิดเจ็ตทับซ้อนกัน ง. จัดเรียงวิดเจ็ตแบบสุ่ม
3. Row Widget มีแกนหลัก (Main Axis) อยู่ในทิศทางใด
4. Stack Widget ใช้เพื่อ
5. MainAxisAlignment.spaceEvenly หมายถึง
6. CrossAxisAlignment.stretch มีผลอย่างไร
7. อธิบายความแตกต่างระหว่าง Column และ Row Widget พร้อมยกตัวอย่างโค้ดการใช้งาน อธิบายการใช้ VerticalDirection ใน Column Widget และผลลัพธ์ที่ได้เมื่อกำหนดค่าเป็น VerticalDirection.up
8. อธิบายการใช้ MainAxisAlignment.center ใน Column Widget และผลต่อการจัดวางวิดเจ็ต เฉลย: จะทำให้วิดเจ็ตลูกทั้งหมดอยู่ตรงกลางของแกนหลัก (แนวตั้ง)
9. อธิบายการใช้ SizedBox ในการจัดระยะห่างระหว่างวิดเจ็ต พร้อมยกตัวอย่างโค้ด
10. อธิบายการใช้ Stack Widget ในการจัดวางวิดเจ็ต พร้อมยกตัวอย่างสถานการณ์ที่เหมาะสม เฉลย: Stack ใช้จัดวางวิดเจ็ตทับซ้อน เช่น การวางข้อความบนรูปภาพ หรือการทำปุ่ม Overlay

LAB 4-6 การพัฒนาแอปพลิเคชันเชิงปฏิบัติ (LAB03-LAB06)

1. อธิบายผลกระทบหากแก้ไขคลาสหลักใน main.dart แต่ไม่แก้ไข widget_test.dart
2. อธิบายการใช้ Scaffold และผลที่เกิดขึ้นหากไม่กำหนด AppBar
3. อธิบายผลลัพธ์หาก Column ไม่กำหนด children
4. อธิบายข้อดีของการแยกคลาสย่อยในการพัฒนาแอปพลิเคชัน
5. อธิบายการทำงานของปุ่ม Random Number และผลลัพธ์ที่ได้
6. อธิบายผลที่เกิดขึ้นหาก ElevatedButton ไม่กำหนด onPressed
7. อธิบายการทำงานของ TextEditingController และประโยชน์ในการควบคุม TextField
8. อธิบายผลกระทบหากไม่แก้ไข pubspec.yaml เพื่อเพิ่ม assets และยกตัวอย่างสถานการณ์จริง

บทที่ 4: การจัดการเหตุการณ์และการรับข้อมูลจากผู้ใช้

- อธิบายความสำคัญของการจัดการเหตุการณ์จากผู้ใช้ต่อประสิทธิภาพของแอปพลิเคชัน หากไม่มีการจัดการที่ดีจะเกิดผลกระทบอะไรบ้าง พร้อมยกตัวอย่างโค้ดหรือสถานการณ์จริง
- วิเคราะห์ข้อจำกัดของการรับข้อมูลผ่านแป้นพิมพ์ และเสนอแนวทางแก้ไข
- อธิบายข้อดีและข้อจำกัดของการรับข้อมูลผ่านเมาส์ พร้อมยกตัวอย่างการใช้งานจริง
- อธิบายข้อดีและข้อจำกัดของการรับข้อมูลผ่านหน้าจอสัมผัส พร้อมยกตัวอย่างการใช้งานจริง
- อธิบายข้อดีและข้อจำกัดของการรับข้อมูลผ่านเสียง พร้อมยกตัวอย่างการใช้งานจริง

1. เป้าหมายของ การทำ แอปพลิเคชันคือการรับข้อมูลจากผู้ใช้ เช่น การคลิกปุ่ม และ จอ เพื่อใช้ ระบบทำงานตามความต้องการของผู้ใช้ ได้ถูกต้อง

✓ หากจัดการเหตุการณ์ได้ดี แอปจะทำงานง่าย ลื่น ลดข้อผิดพลาด

X หากจัดการเหตุการณ์ได้ไม่ดี อาจทำให้แอปตอบสนองช้า ผู้ใช้สับสน ไม่สามารถใช้งานบางฟังก์ชันได้

ตัวอย่างสถานการณ์ เมื่อผู้ใช้กดปุ่ม 'บันทึกข้อมูล' ระบบควรตรวจสอบข้อมูล และ แสดงผลว่าบันทึกสำเร็จ หากไม่มีการจัด Event ที่ดี ปุ่มอาจไม่ทำงานหรือข้อมูลไม่ถูกบันทึก

(หน้า 17-22)

2 การรับข้อมูลผ่านแป้นพิมพ์ (Keyboard Input) เป็นวิธีที่ผู้ใช้ป้อนข้อมูลด้วย ตัวอักษร ตัวเลข หรือสัญลักษณ์ เช่น การกรอกชื่อ รหัสผ่าน

ข้อจำกัด :

- ผู้ใช้อาจพิมพ์ข้อมูลผิด
- ใช้เวลาในการป้อนข้อมูลมากกว่าวิธีอื่นจาก เมนู
- ไม่เหมาะ กับอุปกรณ์ ที่ หน้าจอนำจอจำกัด

แนวทางแก้ไข :

- เพิ่มระบบ ตรวจสอบข้อมูล
- แสดงคำแนะนำหรือ Place holder
- ใช้ Dropdown หรือ Auto Complete ช่วยลดการพิมพ์

(Week 5 หน้า 4)

3 คือ การโต้ตอบกับระบบด้วยการ คลิก เลื่อน ลาก วาง

ข้อดี :

- ใช้งานง่าย
- เหมาะกับผู้ใช้ทั่วไป

ข้อจำกัด :

- ต้องมีอุปกรณ์ชี้ตำแหน่ง

ตัวอย่าง การคลิกปุ่ม Login หรือ ลากไฟล์ เข้าไฟล์เดอร์

(Week 5 หน้า 5)

4 คือ การที่ผู้ใช้สัมผัสหน้าจอโดยตรงเพื่อสั่งงาน เช่น แตะ , ขยับขยาย

ข้อดี :

- สะดวก , รวดเร็ว

ข้อจำกัด :

- ความแม่นยำต่ำกว่าการใช้เมาส์ โยจางาน

ตัวอย่าง การแตะเลือกเมนูในแอปธนาคาร

(Week 5 หน้า 6)

5 คือ การรับคำสั่ง เสียงพูดของผู้ใช้โดยใช้ เทคโนโลยีรู้จำเสียง

ข้อดี :

- สะดวก ไม่ต้องพิมพ์

ข้อจำกัด :

- อาจมีเสียงรบกวนรบกวน

ตัวอย่าง ใช้เสียงสั่งงาน Siri

(Week 5 หน้า 7)

6. อธิบายความแตกต่างระหว่าง Notification และ Dialog ในการสื่อสารกับผู้ใช้ พร้อมยกตัวอย่างการใช้งานจริง
7. อธิบายการใช้ Notification แบบชั่วคราว (Snackbar/Toast) และผลกระทบต่อประสบการณ์ผู้ใช้
8. อธิบายข้อดีของการตรวจสอบข้อมูลฝั่งผู้ใช้ (Client-side Validation) และผลกระทบต่อประสิทธิภาพระบบ

6

รายการเปรียบเทียบ	Notification	Dialog
รบกวนผู้ใช้	ต่ำ	สูง
ต้องการตอบสนอง	ไม่จำเป็น	จำเป็น
วัตถุประสงค์	แจ้งสถานะ	ยืนยัน/รับข้อมูล
• ตัวอย่าง	แจ้งเตือนความสำเร็จ	ยืนยันการลบข้อมูล

(Week 5 หน้า 31)

7

คือ การแสดงข้อความแจ้งเตือนสั้นๆ และหายไปเองโดยไม่ต้องกดปิด

ผลกระทบ :

- ช่วยให้ผู้ใช้เข้าใจว่าระบบกำลังทำอะไร เช่น การบันทึกข้อมูลสำเร็จ หรือเกิดข้อผิดพลาด โดยไม่ขัดจังหวะการใช้งาน

(Week 5 หน้า 29)

8

การตรวจสอบข้อมูลฝั่งผู้ใช้ (Client - Side Validation) คือ การตรวจสอบข้อมูลที่ฝั่ง ผู้ใช้ ก่อนส่งข้อมูลไปยังระบบหลัก

ข้อดี :

ผลกระทบ : ช่วยลดภาระ การประมวลผลของระบบ ทำให้แอปตอบสนองเร็วขึ้น

- ลดข้อผิดพลาดจากข้อมูลที่ผิดๆ
- ผู้ใช้ได้รับผลตอบกลับทันที

(Week 5 หน้า 14 →)

บทที่ 5: การออกแบบฟอร์มและการจัดเก็บข้อมูล

- อธิบายหลักการออกแบบฟอร์มที่ดี และผลกระทบต่อประสบการณ์ผู้ใช้
- ยกตัวอย่างการใช้งานฟอร์มเข้าสู่ระบบ (Login Form) และอธิบายความสำคัญในการตรวจสอบตัวตน
- วิเคราะห์ข้อจำกัดของการจัดเก็บข้อมูลแบบชั่วคราว และเสนอแนวทางแก้ไข
- อธิบายการใช้หน่วยความจำชั่วคราว และผลกระทบเมื่อระบบหยุดทำงานหรือปิดเครื่อง
- อธิบายการใช้ตัวแปรแบบโกลบอล และผลกระทบต่อการทำงานของโปรแกรม
- อธิบายโครงสร้างข้อมูลที่ใช้เก็บหลายค่าในตัวแปรเดียว พร้อมยกตัวอย่างการใช้งานจริง
- อธิบายการจัดเก็บข้อมูลภายในอุปกรณ์ และผลต่อการทำงานของแอปพลิเคชันเมื่อไม่มีอินเทอร์เน็ต
- อธิบายการใช้ Local Storage ในแอปพลิเคชัน พร้อมยกตัวอย่างการใช้งานจริง
- อธิบายการใช้ TinyDB ใน MIT App Inventor สำหรับการจัดเก็บข้อมูลภายใน

1 การออกแบบฟอร์มที่ดี ควรคำนึงถึง ความเรียบง่ายในการใช้งาน , ความชัดเจนของข้อมูล , และลดขั้นตอนที่ไม่จำเป็น
รองรับอุปกรณ์ได้หลากหลาย และ จัดลำดับข้อมูลอย่างเป็นระบบ
ผลกระทบ หาก ฟอร์ม ออกแบบดี ผู้ใช้สามารถกรอกข้อมูลได้รวดเร็ว ลดความผิดพลาด แต่หาก ออกแบบไม่ดี อาจทำให้ผู้ใช้สับสน , กรอกผิด
(Week 5 หน้า 17)

2 ตัวอย่าง แอปธนาคาร แอปชื่อของออนไลน์
ความสำคัญของการตรวจสอบตัวตน รองรับกับ ผู้ที่ไม่ได้รับอนุญาตเข้าถึงข้อมูลส่วนตัว และช่วยรักษาความปลอดภัยของระบบ

3 ข้อจำกัด : แนวทางแก้ไข : ควรใช้การจัดเก็บข้อมูล แบบถาวร เช่น Database , File Storage
• เก็บข้อมูลไว้ยาวไม่ได้
• ข้อมูลสูญหายเมื่อปิดแอป
• ไม่เหมาะ กับข้อมูลสำคัญ
(Week 6 หน้า 4)

4 หน่วยความจำชั่วคราว (Temporary Memory) ใช้สำหรับเก็บข้อมูล ที่จำเป็น ระหว่างการทำงานของโปรแกรม
เมื่อระบบหยุดทำงาน หรือปิดเครื่อง ข้อมูลที่เก็บใน หน่วยความจำชั่วคราว จะถูกลบ ทำให้ข้อมูล ที่ยังไม่ได้บันทึกสูญหาย

* 5 ตัวแปรแบบโกลบอล (Global Variable) คือ ตัวแปรที่สามารถเข้าถึงได้จากหลายส่วนของ โปรแกรม ทำให้สามารถเก็บข้อมูลที่ต้องใช้
งานร่วมกันได้

6 โครงสร้างข้อมูลที่ใช้เก็บหลายค่าในตัวแปรเดียว เช่น List หรือ Array โดยสามารถเก็บข้อมูลหลายรายการไว้ในหน่วยความจำเดียวกัน
ทำให้จัดการข้อมูลจำนวนมากได้ง่ายขึ้น
ตัวอย่าง: การเก็บรายชื่อนักเรียน คะแนนสินค้า หรือรายการสินค้าในร้านค้า

7 การจัดเก็บข้อมูลภายในอุปกรณ์ (Local Storage) คือ การบันทึกข้อมูลไว้ภายในเครื่องของผู้ใช้ เช่น ฐานข้อมูล ไฟล์ หรือระบบจัดเก็บข้อมูล
ภายในเครื่อง
ผลต่อการทำงานของแอปพลิเคชันเมื่อไม่มีเน็ต คือ แอปพลิเคชันสามารถเรียกใช้ข้อมูลได้แม้ไม่มีอินเทอร์เน็ต ทำให้ผู้ใช้สามารถใช้งาน
บางฟังก์ชันแบบ Offline ได้

8 Local Storage คือ การจัดเก็บข้อมูลภายในอุปกรณ์ของผู้ใช้ เพื่อให้ข้อมูลยังคงอยู่แม้ปิดแอปพลิเคชันแล้ว
ตัวอย่าง: แอป Shopping บันทึกรายการสินค้าในตะกร้าไว้ภายในเครื่อง เพื่อให้ผู้ใช้กลับมาเปิดดูได้อีกครั้ง

9 TinyDB เป็นฐานข้อมูลภายใน MIT App Inventor ที่ใช้สำหรับจัดเก็บข้อมูลแบบถาวรภายในอุปกรณ์ โดยข้อมูลจะยังคงอยู่แม้ปิดแอปพลิเคชัน

บทที่ 6: การจัดวางองค์ประกอบ UI และ Layout

1. อธิบายหลักการออกแบบ UI โดยใช้ White Space และผลกระทบต่อประสบการณ์ผู้ใช้
2. อธิบายการใช้ Column Widget ในการจัดเรียงวิดเจ็ต พร้อมยกตัวอย่างการใช้งานจริง
3. อธิบายความหมายของ MainAxisAlignment.spaceEvenly และ CrossAxisAlignment.stretch ใน Column Widget พร้อมยกตัวอย่างการใช้งานจริง
4. อธิบายความหมายของ CrossAxisAlignment.end และผลต่อการจัดวางวิดเจ็ต
5. อธิบายการใช้ Feedback Components เช่น Progress Bar ในการแสดงความคืบหน้า

1 **White Space** คือ พื้นที่ว่างระหว่างองค์ประกอบต่าง ๆ บนหน้าจอ เช่น ระหว่างข้อความ รูปภาพ หรือปุ่มต่าง ๆ การใช้พื้นที่ว่างอย่างเหมาะสมช่วยให้ UI มีความเป็นระเบียบ อ่านง่าย และทำให้ผู้ใช้สามารถมองเห็นข้อมูลสำคัญได้ชัดเจนขึ้น

ผลกระทบต่อประสบการณ์ผู้ใช้:

- * ช่วยลดความซับซ้อนของหน้าจอ
- * ทำให้ผู้ใช้เข้าใจตำแหน่งและหน้าที่ของแต่ละองค์ประกอบได้ง่าย
- * เพิ่มความสวยงามและความสะดวกในการใช้งาน

หากไม่มีการจัด White Space ที่ดี อาจทำให้หน้าจอดูรก ผู้ใช้สับสน และใช้งานได้ยาก

2 **Column Widget** เป็น Widget ใน Flutter ที่ใช้สำหรับจัดเรียง Widget ลูกในแนวตั้ง (Vertical Direction) โดย Widget ต่าง ๆ จะถูกกำหนดไว้ภายใน property children ซึ่งมีรูปแบบเป็น List<Widget>

ตัวอย่างการใช้งาน:

การสร้างหน้าสมัครสมาชิกที่ต้องเรียงช่องกรอกข้อมูล เช่น

- * ช่องกรอกชื่อ
- * ช่องกรอกอีเมล
- * ปุ่มสมัครสมาชิก

"ตัวง่ามวิคัล" →

```
Column(  
  children: [  
    Text("Name"),  
    TextField(),  
    ElevatedButton(  
      onPressed: () {},  
      child: Text("Submit"),  
    ),  
  ],  
)
```

3 **MainAxisAlignment.spaceEvenly** เป็นการจัดวาง Widget ลูกบนแกนหลัก โดยแบ่งพื้นที่ว่างระหว่าง Widget ให้เท่ากัน รวมถึงพื้นที่ด้านบนและด้านล่าง

ตัวอย่าง: ใช้จัดปุ่มเมนูหลายปุ่มในหน้าหลักของแอปให้มีระยะห่าง

CrossAxisAlignment.stretch คือ เป็นการขยาย Widget ลูกให้เต็มพื้นที่ในแกนตรงข้าม

สำหรับ Column ซึ่งมีแกนหลักเป็นแนวตั้ง Cross Axis จะเป็นแนวนอน ทำให้ Widget ขยายเต็มความกว้างของหน้าจอ

ตัวอย่าง: ทำปุ่ม Login ให้มีความกว้างเต็มหน้าจอ

4 **CrossAxisAlignment.end** คือ การจัดตำแหน่ง Widget ลูกให้อยู่ด้านท้ายของแกนตรงข้ามใน Column Widget จะทำให้ Widget ถูกจัดชิดด้านขวาของหน้าจอ เนื่องจากแกนตรงข้ามของ Column คือแนวนอน

ผลลัพธ์: Widget ที่อยู่ภายใน Column จะถูกเลื่อนไปชิดขอบด้านขวา

5 **Feedback Components** คือ ส่วนประกอบ UI ที่ใช้แสดงสถานะหรือผลลัพธ์ของการทำงานให้ผู้ใช้ทราบ เช่น Progress Bar Progress Bar ใช้แสดงความคืบหน้าของกระบวนการที่ใช้เวลา เช่น

- * การดาวน์โหลดไฟล์
- * การโหลดข้อมูลจากระบบ
- * การประมวลผลข้อมูล

6. อธิบายการใช้ `BoxShape.circle` ใน `Container` และผลลัพธ์ที่ได้
7. อธิบายการเปลี่ยน `scrollDirection` เป็น `Axis.horizontal` และผลต่อการเลื่อนหน้าจอ
8. อธิบายการใช้ `Row` ซ้อนใน `Column` และประโยชน์ในการจัดวางวิดเจ็ตหลายแนวพร้อมกัน

6 `BoxShape.circle` เป็นการกำหนดรูปร่างของ `Container` ให้เป็นรูปร่างกลม โดยใช้ร่วมกับ `BoxDecoration`

ผลลัพธ์:

`Container` จะเปลี่ยนจากรูปสี่เหลี่ยมเป็นรูปร่างกลม สามารถใช้สร้าง UI เช่น

- * รูปโปรไฟล์ผู้ใช้
- * ปุ่มวงกลม
- * ไอคอน

7 การกำหนด `scrollDirection: Axis.horizontal` จะเปลี่ยนทิศทางการเลื่อนของ `Widget` จากแนวตั้งเป็นแนวนอน

ผลลัพธ์: ผู้ใช้สามารถเลื่อนเนื้อหาจากซ้ายไปขวา หรือขวาไปซ้ายได้

8 การใช้ `Row` ซ้อนใน `Column` คือ การนำ `Row Widget` มาเป็น `Widget` ลูกภายใน `Column` เพื่อจัดวางองค์ประกอบทั้งแนวตั้งและแนวนอนร่วมกัน

ประโยชน์:

- * สามารถสร้าง `Layout` ที่ซับซ้อนได้
- * จัดวางข้อมูลหลายรูปแบบ ในหน้าจอเดียวกัน

บทที่ 7: (Column, Row, Stack Widget)

1. Column Widget ใช้สำหรับอะไร
2. วิดเจ็ตทับซ้อนกัน ง. จัดเรียงวิดเจ็ตแบบสุ่ม
3. Row Widget มีแกนหลัก (Main Axis) อยู่ในทิศทางใด
4. Stack Widget ใช้เพื่อ
5. MainAxisAlignment.spaceEvenly หมายถึง
6. CrossAxisAlignment.stretch มีผลอย่างไร
7. อธิบายความแตกต่างระหว่าง Column และ Row Widget พร้อมยกตัวอย่างโค้ดการใช้งาน อธิบายการใช้ VerticalDirection ใน Column Widget และผลลัพธ์ที่ได้เมื่อกำหนดค่าเป็น VerticalDirection.up

1 **Column Widget** ใช้สำหรับจัดเรียง Widget ลูกหลายตัวในแนวดิ่ง (Vertical Direction) โดย Widget ที่อยู่ภายใน Column จะถูกกำหนดผ่าน property children ซึ่งมีรูปแบบเป็น List<Widget>

2 ใช้ Stack Widget สำหรับจัดวาง Widget หลายตัวให้ทับซ้อนกัน โดย Widget ที่อยู่หลังสามารถเป็นพื้นหลัง และ Widget อื่นสามารถวางซ้อนด้านบนได้

3 Row Widget มีแกนหลัก (Main Axis) อยู่ในแนวนอน (Horizontal Direction)

4 Stack Widget ใช้สำหรับจัดวาง Widget แบบซ้อนทับกัน โดย Widget ลูกแต่ละตัวสามารถอยู่ตำแหน่งเดียวกันได้

5 **MainAxisAlignment.spaceEvenly** คือ การจัดตำแหน่ง **Widget** ลูกบนแกนหลัก โดยแบ่งพื้นที่ว่างระหว่าง **Widget** รวมถึงพื้นที่ด้านหน้าและด้านหลังให้เท่ากัน

6 CrossAxisAlignment.stretch ทำให้ Widget ลุขขยายเต็มพื้นที่ของแกนตรงข้าม

7 Column Widget ใช้จัดเรียง Widget ในแนวตั้ง Row Widget ใช้จัดเรียง Widget ในแนวนอน

ตัวอย่าง :

Column(

children: [ជនងាយ]

Text("Name"), Name

Text("Email"), Email

1

)

ตัวอย่าง :

Row(

Row(

children: [

```
Icon(Icons.person),
```

Text("User")

1

)

VerticalDirection ใน Column ใช้กำหนดทิศทางการเรียง Widget ในแนวดิ่ง

* VerticalDirection.down เรียงจากบนลงล่าง (ค่าปกติ)

* **VerticalDirection.up** เรียงจากล่างขึ้นบน

เมื่อกำหนดเป็น **VerticalDirection.up** Widget แรกจะอยู่ด้านล่าง และ Widget ถัดไปจะเรียงขึ้นด้านบน

8. อธิบายการใช้ `MainAxisAlignment.center` ใน `Column Widget` และผลต่อการจัดวางวิดเจ็ต เฉลย: จะทำให้วิดเจ็ตลูกทั้งหมดอยู่ตรงกลางของแกนหลัก (แนวดิ่ง)
9. อธิบายการใช้ `SizeBox` ในการจัดระยะห่างระหว่างวิดเจ็ต พร้อมยกตัวอย่างโค้ด
10. อธิบายการใช้ `Stack Widget` ในการจัดวางวิดเจ็ต พร้อมยกตัวอย่างสถานการณ์ที่เหมาะสม เฉลย: `Stack` ใช้จัดวางวิดเจ็ตทับซ้อน เช่น การวางข้อความบนรูปภาพ หรือการทำปุ่ม `Overlay`

8 `MainAxisAlignment.center` ใช้สำหรับจัด `Widget` ลูกให้อยู่ตรงกลางของแกนหลัก ใน `Column` ซึ่งมีแกนหลักเป็นแนวดิ่ง จะทำให้ `Widget` ทั้งหมดอยู่ตรงกลางของหน้าจอในแนวดิ่ง

9 `SizeBox` เป็น `Widget` ที่ใช้กำหนดขนาดหรือสร้างระยะห่างระหว่าง `Widget` เช่น เพิ่มช่องว่างระหว่างปุ่มหรือข้อความ

ตัวอย่าง : `Column(` ผลลัพธ์ คือ มีระยะห่าง 20 pixels ระหว่าง `Text` และ
`children:[` `TextField`
`Text("Username"),`
`SizeBox(height:20),`
`TextField()`
`]`
`)`

10 `Stack Widget` ใช้สำหรับจัดวาง `Widget` หลายตัวให้อยู่ในตำแหน่งเดียวกันหรือทับซ้อนกัน โดย `Widget` ที่ถูกประกาศก่อนจะอยู่ด้านหลัง และ `Widget` ที่ประกาศภายหลังจะอยู่ด้านหน้า

สถานการณ์ที่เหมาะสม:

- * การวางข้อความบนรูปภาพ
- * การสร้างปุ่ม `Overlay`
- * การแสดงไอคอนแจ้งเตือนบนรูปโปรไฟล์

LAB 4-6 การพัฒนาแอปพลิเคชันเชิงปฏิบัติ (LAB03-LAB06)

1. อธิบายผลกระทบหากแก้ไขคลาสหลักใน main.dart แต่ไม่แก้ใน widget_test.dart
2. อธิบายการใช้ Scaffold และผลที่เกิดขึ้นหากไม่กำหนด AppBar
3. อธิบายผลลัพธ์หาก Column ไม่กำหนด children
4. อธิบายข้อดีของการแยกคลาสน้อยในการพัฒนาแอปพลิเคชัน
5. อธิบายการทำงานของปุ่ม Random Number และผลลัพธ์ที่ได้
6. อธิบายผลที่เกิดขึ้นหาก ElevatedButton ไม่กำหนด onPressed

1 จะทำให้เกิดข้อผิดพลาดในการทดสอบ (Test Error) เนื่องจากไฟล์ทดสอบยังอ้างอิงชื่อคลาสเดิม ที่ไม่มีอยู่แล้ว

ผลกระทบ :

- โปรแกรมทดสอบไม่สามารถทำงานได้
- เกิด error ตอน run

2 Scaffold เป็นโครงสร้างหลักของหน้าจอใน Flutter ให้ส่วนจัดองค์ประกอบ พื้นฐาน เช่น AppBar Body Floating Action Button Drawer

หากไม่กำหนด AppBar แอปยังสามารถทำงานได้ปกติ แต่จะไม่มีแถบด้านบนของหน้าจอ

3 Column ใช้สำหรับเรียง Widget หลายตัวในแนวตั้ง โดย Widget ที่ต้องการแสดงจะต้องกำหนดใน property children

หากไม่กำหนด children Column จะไม่มี Widget ภายใน ทำให้หน้าจอไม่แสดงองค์ประกอบใดๆ

4 ข้อดี ♥

- ทำให้โค้ดเป็นระเบียบ
- ง่ายต่อการแก้ไข , พัฒนาเพิ่ม
- นำ Widget มาใช้ทำได้
- ลดความซ้ำซ้อนของโปรแกรม

5 ปุ่ม Random Number ให้ส่วนสร้างตัวเลขแบบสุ่มเมื่อผู้ใช้กดปุ่ม โดย Event ของปุ่ม โดย Event ของปุ่มจะเรียกคำสั่ง Random เพื่อสร้างค่าใหม่ และนำค่าที่ได้ไปแสดงบนหน้าจอ

การทำงานของ : 1. ผู้ใช้กดปุ่ม

2. ระบบเรียกฟังก์ชันสุ่มตัวเลข

3. นำค่าที่ได้มาแสดงบน Text Widget

6 onPressed เป็น Event Handler ที่กำหนดการทำงานเมื่อผู้ใช้กดปุ่ม

หากไม่กำหนด onPressed ปุ่มจะอยู่ในสถานะ Disable ทำให้ผู้ใช้ไม่สามารถกดเพื่อเรียกคำสั่งได้

7. อธิบายการทำงานของ TextEditingController และประโยชน์ในการควบคุม TextField
8. อธิบายผลกระทบหากไม่แก้ไข pubspec.yaml เพื่อเพิ่ม assets และยกตัวอย่างสถานการณ์จริง

7 Text Editing Controller เป็นตัวควบคุมข้อมูลภายใน Text Field ใช้สำหรับอ่านค่า แก้ไข หรือ จัดการ ข้อ ความ ที่ผู้ใช้ป้อนเข้ามา
ประโยชน์ :

- ดึงข้อความจาก Text Field ไปประมวลผลได้
- กำหนดค่าเริ่มต้นให้ช่องกรอกข้อมูลได้
- ล้างข้อความในช่องกรอกข้อมูลได้

8 หากไม่ได้เพิ่ม assets ในไฟล์ pubspec.yaml ระบบจะไม่สามารถค้นหาไฟล์เหล่านั้นได้
ตัวอย่าง : ต้องการแสดงรูปโลโก้ในแอป แต่ไม่ได้เพิ่ม path รูปภายใน pubspec.yaml ทำให้รูปไม่สามารถโหลดได้